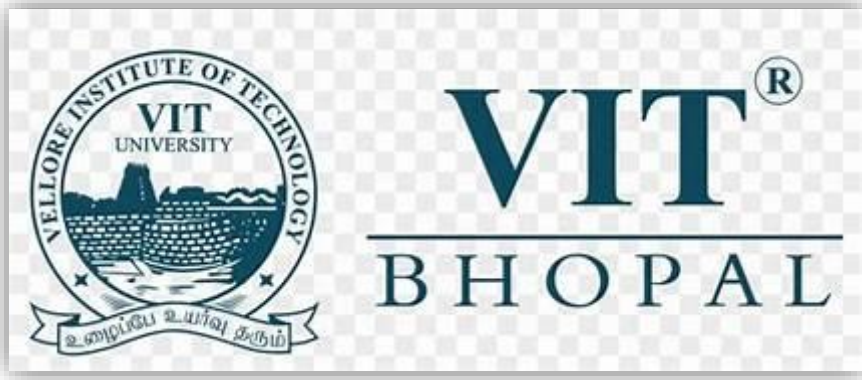




INCOME TAX CALCULATOR

NAME - **SHRADDHA**
REG. NO. - **25BEC10057**
BRANCH - **ECE**
BATCH - **2025-29**
COURSE CODE - **CSE1021**
FACULTY - **DR. A. BALAJI**
DATE - **31-03-2026**

1. Introduction

The **Income Tax Calculator** is a software tool developed using Python. Its primary purpose is to automate the computation of income tax for individuals based on their gross annual income and applicable deductions. By adhering to specific tax slabs and adding necessary cesses, the application simplifies the financial estimation process for users, eliminating the need for manual calculations.

2. Problem Statement

Manual income tax calculation is often prone to human error due to the complexity of progressive tax slabs and the addition of specific cesses. Individuals often struggle to determine exactly which tax bracket they fall into and how the marginal tax rates apply to different portions of their income. The problem this project addresses is the need for a quick, error-free, and automated way to calculate tax liability via a simple command-line interface.

3. Functional Requirements

The system fulfils the following functional requirements based on the implemented code:

- **Input Processing:** The system must accept "Gross Annual Income" and "Deductions" as numeric inputs from the user.
- **Taxable Income Calculation:** The system must automatically subtract deductions from the gross income. If the result is negative, taxable income is set to 0.
- **Slab Application:** The system must apply the following tax rates:
 - 0% for income up to 3,00,000.
 - 5% for income between 3,00,001 and 7,00,000.
 - 10% for income between 7,00,001 and 10,00,000.
 - 15% for income between 10,00,001 and 12,00,000.
 - 20% for income between 12,00,001 and 15,00,000.
 - 30% for income above 15,00,000.
- **Rebate Handling:** If the taxable income is less than or equal to ₹7,00,000, the tax must be reduced to 0.
- **Cess Calculation:** A 4% Health and Education Cess must be added to the calculated base tax.

4. Non-functional Requirements

- **Robustness:** The system implements error handling (try-except) to manage non-numeric inputs gracefully without crashing.
- **Usability:** The interface is a clean CLI (Command Line Interface) with clear prompts and separators for readability.
- **Accuracy:** Floating-point calculations are used to ensure precision in monetary values.

5. System Architecture

The application follows a **Monolithic, Procedural Architecture**.

- **Input Layer:** Collects raw data from the standard input (keyboard).
- **Logic Layer:** Contains the business rules (Tax Slabs, Rebates, Cess logic).
- **Output Layer:** Formats and prints the results to the console.

6. Design Decisions & Rationale

- **Language Selection (Python):** Python was chosen for its simplicity in handling mathematical operations and string formatting, making the code easy to read and maintain.
- **Conditional Structure (if-elif-else):** This structure was selected to implement the progressive tax slabs because the ranges are mutually exclusive and hierarchical.
- **Rebate Logic Placement:** The check for the ₹7,00,000 rebate is placed *after* the initial slab calculation but *before* the cess calculation to ensure users in this bracket pay zero tax effectively.

7. Implementation Details

The core logic relies on mathematical formulas implemented in Python.

Key Formulas:

1. Taxable Income:

$\text{taxable} = I_{\text{gross}} -$

2. Cess Calculation:

$\text{Cess} = \text{Tax}_{\text{base}} \times 0.04$

3. Total Tax:

$\text{Tax}_{\text{total}} = \text{Tax}_{\text{base}} + \text{Cess}$

Code Snippet (Input Handling):

Python

try:

```
gross_income = float(input("Enter Gross Annual Income: "))
```

```
deductions = float(input("Enter Deductions...: "))
```

except ValueError:

print("Invalid input. Please enter numbers only.")

exit()

8. Screenshots / Results

CODE:

```
D:\Projects\Income-Tax-Calculator > tax_calc.py ...
1 print("--- Simple Income Tax Calculator ---")
2 try:
3     gross_income = float(input("Enter Gross Annual Income: "))
4     deductions = float(input("Enter Deductions (Standard Deduction is usually 75000): "))
5 except ValueError:
6     print("Invalid input. Please enter numbers only.")
7     exit()
8
9 taxable_income = gross_income - deductions
10
11 if taxable_income < 0:
12     taxable_income = 0
13 print("\nTaxable Income: " + str(taxable_income))
14
15 tax = 0
16
17 if taxable_income <= 300000:
18     tax = 0
19
20 elif taxable_income <= 700000:
21     tax = (taxable_income - 300000) * 0.05
22
23 elif taxable_income <= 1000000:
24     tax = 20000 + (taxable_income - 700000) * 0.10
25
26 elif taxable_income <= 1200000:
27     tax = 50000 + (taxable_income - 1000000) * 0.15
28
29 elif taxable_income <= 1500000:
30     tax = 80000 + (taxable_income - 1200000) * 0.20
31
32 else:
33     tax = 140000 + (taxable_income - 1500000) * 0.30
34
35 if taxable_income <= 700000:
36     tax = 0
37
38 cess = tax * 0.04
39 total_tax = tax + cess
40
41 print("-" * 30)
42 print(f"Base Tax:      {tax:.2f}")
43 print(f"Cess (4%):      {cess:.2f}")
44 print("-" * 30)
45 print(f"Total Payable: {total_tax:.2f}")
46 print("\n" * 30)
```

INPUT

```
PS D:\Projects\Library-Management-System> & "C:/Program Files/PYTHON/python.exe" d:/Projects/Income-Tax-Calculator/tax_calc.py
--- Simple Income Tax Calculator ---
Enter Gross Annual Income: 2900000
Enter Deductions (Standard Deduction is usually 75000): 7689
```

OUTPUT

```
Taxable Income: 2892311.0
```

```
-----  
Base Tax:      557693.30
```

```
Cess (4%):     22307.73
```

```
-----  
Total Payable: 580001.03
```

```
-----  
PS D:\Projects\Library-Management-System> █
```

9. Testing Approach

- **Unit Testing (Manual):**
 - **Test Case 1 (Rebate):** Input Income: 7,50,000, Deductions: 50,000. *Expected Result:* Taxable 7,00,000 -> Tax 0.
 - **Test Case 2 (High Income):** Input Income: 16,00,000, Deductions: 0. *Expected Result:* 30% slab application on amount > 15L.
 - **Test Case 3 (Invalid Input):** Input "Ten Thousand". *Expected Result:* "Invalid input" message and graceful exit.

10. Challenges Faced

- **Logic Flow for Rebate:** Ensuring that the rebate for income under ₹7 Lakhs was applied correctly. Initially, the calculation might have applied the 5% slab, but a specific override was required (if taxable_income <= 700000: tax = 0) to meet tax regime rules.
- **Input Validation:** Preventing the program from crashing when non-numeric characters were entered. This was solved using try-except blocks.

11. Learnings & Key Takeaways

- **Conditional Logic:** Deepened understanding of how to map real-world tiered rules (tax slabs) into programming logic structures.
- **Data Types:** Importance of using float for financial calculations to accommodate decimal values.
- **User Experience:** Learned to format output strings (e.g., f"{tax:.2f}") to make financial data readable.

12. Future Enhancements

- **GUI Implementation:** Developing a Graphical User Interface using Tkinter or a web-based interface using Flask.
- **Regime Selection:** Adding a feature to allow users to switch between the "Old Tax Regime" and "New Tax Regime" to compare benefits.
- **PDF Export:** Implementing functionality to save the tax breakdown as a PDF or Text file for record-keeping.

13. References

- **Python Documentation:** For standard input/output and control flow syntax.
- **Project Source Code:** `tax_calc.py` and `README.md` included in the repository.